Product Requirements Document (PRD)

Project Unified Finance: Custom Single-Couple Financial Framework

Document Version: 1.2 (Dynamic Sheets Taxonomy Implementation)

Author Role: Vibe Coder / Product Architect

Target Runtime: Responsive Single Page App (SPA)

Backend Database: Google Sheets API v4 Dynamic Schema Store

1. System Architecture & Scope Boundary

Project Unified Finance is a highly tailored, low-friction financial orchestrator designed exclusively for a single couple managing unified finances. The technical architecture relies on an elegant decoupling strategy: a bespoke frontend optimized for mobile web interactions paired with a structural Google Sheets API backend. This architecture guarantees absolute data ownership, zero database hosting costs, and standard spreadsheet elasticity, combined with an elite, professional-grade user experience layer.

1.1 Scope Boundaries & Non-Functional Requirements

- **Multi-Tenancy:** Out of Scope. No multi-tenant partitioning, user registration flows, or complex Role-Based Access Control (RBAC) layers shall be implemented. The system operates as a single-family runtime deployment.
- **Authentication:** Deferred. To minimize transactional friction, traditional user login sequences are omitted from the initial deployment phase. The web application operates on a secured, obfuscated deployment URL.
- **Performance:** Data payload sizes remain lightweight. API requests to read/write spreadsheet data must balance eagerly through async UI states, maintaining a crisp, interactive frame rate.

2. Google Sheets Database Schema (Greenfield Implementation)

The engineer must programmatically initialize an entirely new Google Spreadsheet workbook. The configuration must contain six explicitly structured operational sheets (tabs). Every data ingestion pipeline must automatically generate a cryptographically sound unique identifier string (UUID) per transaction block to serve as a primary record key.

2.1 Tab Layout: `transactions`

Captures historical cash outflows across budget parameters.

Column Name	Data Type	Validation Logic & Field Constraints
`id`	String (UUIDv4)	Unique, client-side or server-side generated primary tracking key.
`date`	Date (`YYYY-MM-DD`)	ISO-standard calendar execution date. Mandatory.
`group`	String	The parent expense category matching dynamically parsed system values. See Section 6.
`subgroup`	String	The explicit sub-category child element filtered by dynamic parent bounds. See Section 6.
`amount`	Decimal (`0.00`)	Absolute transaction numerical value. Fixed to two decimal thresholds.
`description`	String (Nullable)	Optional free-text data. If populating content, flags the UI info popover.

2.2 Tab Layout: `income_ledger`

Records all unified cash inflows generated by the family unit.

Column Name	Data Type	Validation Logic & Field Constraints
`id`	String (UUIDv4)	Primary tracking key. Unique.
`date`	Date (`YYYY-MM-DD`)	Calendar ingestion date. Mandatory.
`source`	String	The income source entity. Governed by dynamic configuration mappings under the "Income" group column.
`amount`	Decimal (`0.00`)	Numerical value of incoming funds. Fixed currency precision.
`description`	String (Nullable)	Optional text notes concerning the specific revenue influx event.

2.3 Tab Layout: `savings_accounts`

Manages long-term cash reservations, goals, and allocation logs.

Column Name	Data Type	Validation Logic & Field Constraints
`account_id`	String	Unique lookup slug for specific savings targets.
`account_name`	String	Custom designated name of the savings vehicle or bucket.
`target_amount`	Decimal (Nullable)	Optional target value representing absolute completion threshold.
`target_date`	Date (Nullable)	Optional deadline matrix to evaluate pacing models.
`date_logged`	Date (`YYYY-MM-DD`)	The transaction timestamp mapping the exact allocation date.
`amount_added`	Decimal (`0.00`)	The specific funding allocation block written to the bucket.

2.4 Tab Layout: `debt_ledger`

Tracks dynamic outstanding external claims including obligations to or receivables from third parties.

Column Name	Data Type	Validation Logic & Field Constraints
`id`	String (UUIDv4)	Primary record identification key.
`type`	Enum String	Strict constraint matching only `Payable` or `Receivable`.
`description`	String	Explains counterparty information and transaction core rationale.
`total_amount`	Decimal (`0.00`)	The baseline principal balance written or remaining.
`date_logged`	Date (`YYYY-MM-DD`)	Timestamp tracking initialization vectors.

2.5 Tab Layout: `monthly_configs`

Houses system goal structures and allocation maps configured via the Month Plan user panel.

Column Name	Data Type	Validation Logic & Field Constraints
`month_year`	String (`MM-YYYY`)	The specific calendar month block scope target.
`config_type`	String	Categorization key tracking rules like `Budget_Limit` or `Income_Goal`.
`key_name`	String	The specific grouping label or entity target string (e.g. "Groceries", "Salary").
`allocated_value`	Decimal (`0.00`)	The target monetary limit or objective benchmark for the designated cycle.

2.6 Tab Layout: `categories_config`

The system taxonomy configuration engine. The frontend web app parses this matrix dynamically to generate drop-down selectors. Users modify columns and rows inside this sheet to alter categories without changing application source code.

```
[VISUAL LAYOUT SCHEMA IN THE GOOGLE SHEET TAB]
Row 1 (Headers):    [Group_A]          [Group_B]          [Group_C]      ... [Group_L]
Row 2 (Data):       Subgroup_A1        Subgroup_B1        Subgroup_C1    ... Subgroup_L1
Row 3 (Data):       Subgroup_A2        Subgroup_B2        Subgroup_C2    ... Subgroup_L2
Row 4 (Data):       Subgroup_A3        (Empty Cell)       Subgroup_C3    ... (Empty Cell)
```

3. Organic Professional Design System Specification

The interface rejects generic, high-saturation technology colors in favor of a customized, "quiet luxury" organic layout. This aesthetic balances earthy tones, rich light canvases, and geometric fonts to turn financial tooling into an enjoyable, highly intuitive environment.

3.1 Aesthetic Color Tokens

Design Token	Hex Code	System Implementation & Interface Mapping
Primary Accent (Deep Pine)	`#1E352F`	Primary header fills, focus state actions, callout metrics, active text blocks.
Secondary Accent (Sage/Olive)	`#4A5D4E`	Secondary action states, trend lines, subheaders, structural framing lines.
Base Canvas (Rich Cream)	`#FDFBF7`	Global application background canvas. Soft, low contrast, highly scannable.
Surface Variant (Warm Sand)	`#F4EFE3`	Dashboard blocks, card units, table body alternate stripes, interactive states.
Primary Font (Dark Walnut)	`#2B2421`	High-density alphanumeric characters, active numerals, primary navigation text.
Secondary Font (Soft Earth)	`#6E6560`	Meta labels, dates, placeholder data strings, helper text blocks.
System Alert (Crimson Glow)	`#A34843`	Validation structural failures, negative accounts, debit targets, error borders.

3.2 Typography & Formatting Constraints

- **Font Families:** The primary system font is `Inter` or `Plus Jakarta Sans`. Fallback stack defaults to standard clean sans-serif layouts.
- **Numerical Tabular Formatting:** All financial values, ledgers, and transaction counters must enforce tabular numbers: `font-variant-numeric: tabular-nums;`. This prevents alignment shifting when numerical data changes.
- **Heading Scale:** Main view titles follow `18pt-22pt (Bold)`. Screen sections follow `13pt-15pt (Semi-Bold)`. Inline elements use `11pt-12pt (Italic/Medium)`.

3.3 Motion & Transition Parameters

Animations must remain highly professional and smooth. Oversaturated, bounce-heavy mobile motions are strictly banned. The engine must implement hardware-accelerated transforms matching the following specs:

> **Screen-to-Screen Horizon Glide:** Switching global nav views must trigger a lateral sliding motion across the screen interface.
>
> *Physics Parameters:* `350ms` execution timeframe utilizing a custom cubic spline acceleration profile: `cubic-bezier(0.16, 1, 0.3, 1)`, ensuring an elegant, decelerating ease-out curve.

> **Block Expansion Mechanics:** Tapping sub-category summary blocks triggers a clean, vertical expansion sequence.
>
> *Physics Parameters:* `300ms` transition with simultaneous layout fade-in for internal charts. Element overflow parameters must remain hidden during compilation to prevent visual stuttering.

4. Core Screen Specifications

4.1 Dashboard View (Default Initial Route)

- **Purpose:** Instant, glanceable appraisal of high-level financial parameters upon launching the system web interface.
- **Core Progress Layout:** Displays high-priority tracking bars evaluating actual current calendar-month performance against goals generated in the *Month Plan*.
  - **Aggregate Budget Progress:** Computes $\sum Amount_{spent}$ divided by total current configuration budget caps.
  - **Aggregate Goal Progress:** Visualizes current tracking velocity towards savings and income benchmarks.

- **Asset Positioning:** Current account liquid balances are styled as clear text cards, each paired with a highly contextual visual icon for rapid, immediate identification.

4.2 Transactions Ledger View

- **Static Aggregation Header:** Displays large, prominent metrics summarizing current month activity: `Total Money In` vs. `Total Money Spent`. These values recalculate eagerly on any transaction write.
- **The Financial Ledger Table:** Renders a reverse-chronological stream of all rows in the `transactions` spreadsheet tab matching the active calendar view boundaries.
- **Column Layout Matrix:** `Date` | `Group` | `Subgroup` | `Amount` | `Info Action`
- **Popover UI Mechanics:** If a row contains data in the `description` string cell, a minimalist info icon appears next to the amount column. Clicking this icon opens a small, centered popover layout showing the plain text string note. Tapping anywhere outside the layout bounds dismisses the view.

4.3 Budget Groups Tracking View

- **Dynamic Block Matrix Initialization:** The system reads row 1 of the `categories_config` sheet at initialization. It dynamically renders a modular display block for every expense column header parsed (excluding the 'Income' column block). Each block features an intuitive linear progress indicator mapping `Spent Current Month` versus the configuration limit values pulled from `monthly_configs`.
- **State Expansion Sequence:** Tapping an individual parent block expands the item layout to reveal deep analytical elements:

  Top Left Component:

  Granular numeric tracking tracking exactly how much capacity remains in the category before breaching the threshold budget limits.

  Top Right Component:

  A native rendering engine pie chart illustrating the internal subgroup percentages contributing to the total expenditure, mapped to an explicit color key index.

- **Filtered Context Ledger:** The bottom region of the expanded block populates a specialized data table containing only transactions matching that parent category name, formatted cleanly as: `Date` | `Subgroup` | `Amount` | `Info Icon Indicator`.

4.4 Savings Accounts Management View

- **Aggregate Header Modules:** Tastefully displays large metrics highlighting the absolute status of savings vehicles:
  1. Total combined cash successfully routed into long-term savings structures during the active calendar month.
  2. Global system tracking progress towards the aggregate master savings goal threshold.
  3. High-emphasis display block highlighting the **Total Combined Savings Balance Across All Vehicles**.
- **Interactive Block Array:** Renders clean surface blocks tracking independent target funds. Clicking a block triggers an expansion workflow showing the specific current net balance alongside a chronological history list of all historic matching inbound allocations: `Date` | `Amount Logged` | `Info Popover State`.

4.5 Income and Payables View

This layout enforces a dual-zone structural framework splitting asset injection tracking from debt settlement variables.

4.5.1 Top Half UI: Income Success Dashboard

- **Celebratory Goal Header:** A large, prominent display tracking total earned income for the current calendar month against the stated objective parameter. Visual styling should highlight milestones and celebrate financial success when performance approaches or crosses the target benchmark.
- **Income Register:** Below the header layout, a clean, dedicated ledger list renders all chronological items pulled directly from the `income_ledger` spreadsheet.

4.5.2 Bottom Half UI: Debts, Receivables, and Payables Tracker

Divided into two distinct visual blocks arranged in a horizontal layout structure:

Left Component: Payables Ledger

Tracks total absolute liabilities owed to outside entities. Progress trackers evaluate the volume of capital dispatched to settle debts within the active month window.

Right Component: Receivables Ledger

Tracks capital owed back to the family unit by third parties. Progress components monitor inbound fulfillment velocity captured within the active calendar month matrix.

> **Critical Persistence & Transition Rules:** Absolute outstanding core debt values carry over across month boundaries indefinitely until completely settled. However, indicators monitoring "Paid This Month" or "Received This Month" flush and reset to exactly `0` on the 1st day of every new calendar month.

4.6 Analytics Insight Engine

- **Segment Control Array:** The landing state presents 4 clean segment control buttons: `Expense`, `Savings`, `Income`, and `Net`.
- **Data Visualization Profile:** Selecting a control state updates a native Line Chart illustrating actual historical performance plotted alongside target goals over time.
- **Analytical Control Ingress:** Quick-toggle control tabs filter data sets by Year-to-Date (YTD), Quarters, or Specific Individual Months. The interface must calculate and overlay dynamic horizontal lines indicating the calculated mathematical averages across the selected timeframe.

4.7 Month Plan Configuration Wizard

- **UX Paradigm:** A step-by-step interactive setup wizard designed to establish budget targets and financial configurations for the upcoming calendar month.
- **Dynamic Taxonomy Fetch:** The configuration parameters populated inside this wizard must map dynamically to the column headers fetched from the `categories_config` sheet, ensuring user-added categories automatically receive setup cards in the workflow sequence.
- **Trend Line Analysis Engine:** Displays an overlay chart detailing actual performance versus goals over the `previous 4 calendar months`.
- **Cold Start Exception Logic:** If the user launches the system and no historical rows populate the spreadsheet database, the trend rendering engine must catch the null dataset exception gracefully. It must map the missing historical tracking nodes cleanly to `0` values on the chart without generating system errors or breaking the interface view. Selecting "Save" writes configurations directly to the `monthly_configs` tab.

5. Global Input Engineering: The "Flex" Process

The input framework is engineered to optimize data entry speed, minimizing friction when adding transactions via mobile devices.

5.1 The Flex Floating Action Button (FAB)

- **Ergonomic Layout Profile:** Scaled and positioned in the extreme bottom-right corner of the viewport interface, matching the average sweep radius of a user's thumb when holding a mobile device.
- **Visual Identifier:** Features a clean "Flex" muscle symbol.
- **Trigger Event Logic:** Tapping the target icon rotates the base element 45 degrees, opening a smooth radial or vertical options list with 4 clear choices:
  `1. Add Expense` | `2. Record Income` | `3. Add Savings Account` | `4. Report Payable/Receivable`.

5.2 Overlays & Navigation Controls

- **Viewport Configuration:** Activating an input option launches a full-screen overlay (`100vw / 100vh` visual bounds) masking the background.
- **Keyboard Navigation Flow:** The wizard operates via keyboard-focused logic. Typing `Enter` or `Return` automatically validates the active data input, saves it to the temporary model state, and advances active system focus to the next input field on the screen.

5.3 Form Field Interaction & Validation Logic

```
[FORM PROCESS FLOW: ADD EXPENSE]
├── 1. Date Selector
│    └── User Interface presents two explicit buttons: "Today" automatically sets current
timestamp.
│    └── Mini-calendar icon picker allows selection of alternative past dates.
├── 2. Category Dropdown
│    └── ENGINE INTERACTION: Fetches Row 1 Headers from the `categories_config` spreadsheet
tab.
│    └── Renders parsed column names as options. Selection populates step 3 dynamically.
├── 3. Sub-category Dropdown
│    └── ENGINE INTERACTION: Reads all active rows beneath the chosen group's specific
header column.
│    └── Filters out empty cells; binds remaining tokens as the selectable subgroup options
array.
├── 4. Amount Input
│    └── Enforces ATM right-to-left decimal shift behavior.
│    └── Example sequence: Type '53' -> UI registers $0.53. Type '4' next -> UI registers
$5.34.
└── 5. Description Input
     └── Optional free-text alphanumeric string. Pressing Enter clears focus tracking.

[FORM PROCESS FLOW: RECORD INCOME]
├── 1. Date Selector (Identical to Expense flow rules)
├── 2. Source Input
│    └── ENGINE INTERACTION: Autocompletes matching known rows listed underneath the
'Income' column column of the taxonomy configuration sheet.
│    └── Exception Rule: If submitted source token does not exist in the dynamic column
array:
│         └── Halt processing. Launch blocking modal alert window.
│         └── Query: "Add this source to your permanent source configuration list?"
│         └── [Selected YES]: Append token as a new row under the 'Income' column in
`categories_config`, complete write.
│         └── [Selected NO]: Record entry in ledger under custom source name; omit from
spreadsheet lookup array.
├── 4. Amount Input (Identical ATM right-to-left decimal shift rules)
└── 5. Description Input (Optional free-text string field)
```

5.4 Form Completion Controls & Validation Error States

The base of every record panel displays three fixed interaction buttons:

1. **Discard (Left Ingress Button):** Launches a confirmation modal query. If verified, clears form data and closes the window. If rejected, returns the user to the active input state.

2. **Save and Exit (Center Button):** Validates required data fields. If valid, posts the data string payload to the Google Sheet API, triggers a successful entry sequence (animated green checkmark overlay + audio chime), and returns the user to the default Dashboard view via a smooth glide transition.
3. **Save and Record New (Right Button):** Validates data fields. If valid, posts data to the Google Sheet, triggers the success checkmark animation overlay, clears form inputs, and resets active focus back to the first field index to allow rapid, back-to-back entries.

> **Validation Failure State Protocol:** If mandatory variables are missing when an entry action is triggered, submission halts immediately. The interface launches an alert dialog and applies a **glowing red outer neon shadow effect** around the missing input containers to provide immediate visual feedback.

6. Dynamic Spreadsheet Parsing Pipeline Specifications

To fulfill the requirement that the system taxonomy remain completely user-editable directly through the spreadsheet interface without requiring code revisions, the engineer must implement a matrix transposition engine. At runtime or application boot sequence, the web application executes a cell-range read query targeting the `categories_config` worksheet.

6.1 Parsing Algorithm & Pseudocode

The backend mapping engine must treat column headers as independent structural arrays. The following processing loop outlines the parsing expectation for the engine:

```
FUNCTION initializeDynamicTaxonomy():
    // 1. Query the entire bounding box grid of row records from the tab sheet
    sheetDataMatrix = GoogleSheetsAPI.readRange("categories_config!A1:Z100")

    // 2. Extract column master elements from the first structural row sequence
    masterGroupsArray = sheetDataMatrix[row: 0] // Extracts keys: ["Education",
"Food", ...]

    // 3. Initialize relational dictionary map objects
    SYSTEM_TAXONOMY_MAP = {}

    FOR EACH columnIndex, groupName IN masterGroupsArray:
        SYSTEM_TAXONOMY_MAP[groupName] = []

        // Iterate downwards through rows starting from index 1 to harvest sub-elements
        FOR EACH rowIndex FROM 1 TO sheetDataMatrix.length - 1:
            cellValue = sheetDataMatrix[row: rowIndex, col: columnIndex]

            // Check for valid alphanumeric character blocks; break or skip empty
allocations
            IF cellValue IS NOT NULL AND cellValue.trim() != "":
                SYSTEM_TAXONOMY_MAP[groupName].push(cellValue.trim())

    // 4. Inject mapped object store into frontend context providers
    ApplicationContext.setTaxonomy(SYSTEM_TAXONOMY_MAP)
END FUNCTION
```

6.2 Execution Scenarios & Runtime Validation Boundaries

- **Appending a Group:** If you add a new text column header to cell `M1` (e.g. `Hobbies`) and insert rows underneath it, the app engine will automatically discover the key during parsing. It will instantly generate a

new interactive configuration module inside the *Month Plan* setup screen and a new linear tracking block inside the *Budget Groups* tab without compiling source code edits.

- **Removing an Option:** Clearing a cell or row inside a column instantly drops that token from the frontend dropdown arrays on the next hot reload or app initialization call, guaranteeing absolute structural control directly from your spreadsheet canvas.